A Systematic Review of DevSecOps Integration in CI/CD Pipelines: From Culture to Continuous Security, Challenges, and Measurable Outcomes

Abstract

The rapid adoption of DevOps and Continuous Integration/Continuous Deployment (CI/CD) has transformed software delivery from quarterly releases to multiple deployments per day. However, this acceleration has exposed a critical gap: security, when treated as a bolt-on phase, fails to keep pace with development velocity, resulting in misconfigurations, exposed secrets, and breaches that cost 100x more to fix in production than in code. This review paper systematically analyzes the literature on DevOps, CI/CD, and DevSecOps, structured around a unique with-vs-without security comparative lens derived from core review notes.We deconstruct (1) the foundational relationship between DevOps as culture and CI/CD as practice (middle pipeline), (2) the security-integrated CI/CD methodology versus no-security pipelines, (3) the full DevOps process from Plan to Monitor with embedded security controls versus blind operations, (4) six core DevOps adoption challenges and their proven solutions, (5) the criticality of evaluation methodologies using DORA metrics and SLOs, (6) secrets and configuration management, (7) layered continuous security using Snyk, DAST, and OWASP ASVS, (8) misconfiguration detection as the leading cause of cloud breaches, (9) secure Continuous Deployment concerns, and (10) IAM least privilege and lifecycle management.Furthermore, we analyze five specific research gaps identified in prior literature: CIS Baseline vs. No Detection Rules for Other Services, STRIDE/Risk Analysis vs. No Tool-Based Detection, Tool Integration vs. No Misconfig Detection, Secure vs. Non-Secure CD vs. Misconfigs Not Defined, and Security Monitoring Solution vs. Only Theoretical. We conclude by proposing a modern 2026 blueprint integrating Gitleaks, Semgrep, Trivy, Checkov, OPA/Conftest, Falco, and Prowler with SBOM and SLSA attestation.Findings indicate that organizations adopting shift-left, automated, and continuous security achieve fast, safe, and compliant releases, while those without it experience breaches, fines, downtime, and loss of trust. The difference is not tooling but discipline: measured, automated, and culturally embedded security.

Keywords: DevSecOps, CI/CD Pipeline, DevOps Culture, DORA Metrics, SAST, DAST, SCA, IaC Scanning, Misconfigurations, Policy as Code, CIS Benchmark, STRIDE, Security Monitoring

1. Introduction

In the last decade, software engineering has shifted from Waterfall to Agile to DevOps, where DevOps as defined by Myrbakken & Colomo-Palacios (2017) is not a tool but a culture that unifies Development and Operations through shared responsibility, automation, and continuous feedback with its lifecycle being infinite looping continuously through Plan → Code → Build → Test → Release → Deploy → Operate → Monitor, and CI/CD is often conflated with DevOps but our core review clarifies that DevOps is the culture (who + why), CI/CD is the practice (what + how), and CI/CD lives inside DevOps as one of its core automation lanes, where DevOps spans the full ∞ lifecycle while CI/CD covers only the middle pipeline of Code → Build → Test → Deploy and DevOps includes additional lanes such as Infrastructure as Code (IaC), Site Reliability Engineering (SRE), DevSecOps, and Monitoring, so that without DevOps, CI/CD is a pipeline without context and without CI/CD, DevOps is a culture without execution, as illustrated in Figure 1: DevOps (Culture) encompasses CI/CD (Practice) → plus IaC + SRE + DevSecOps + Monitoring, and with DevOps + CI/CD teams ship fast, safe, and happy while without them releases are slow, manual, risky, and siloed (Ivanov & Smolander, 2018; Arachchi & Perera, 2018), and Singh (2020) notes that CI/CD pipelines deploy multiple times per day leaving no room for manual security reviews while Mangla (2023) empirically demonstrates this where a non-secure pipeline detected 0 misconfigurations while a secure pipeline detected 155 IaC flaws and 9 Critical + 11 High cloud posture violations and Williams and Wichers (2019) quantify the economics where early detection reduces remediation cost by up to 30 times and in production the cost can be 100x more, and this review is motivated by the need to systematically compare with security (DevSecOps) versus no security across every DevOps phase, challenge, and tool using the core review notes as a structured framework.

2. Research Methodology

Structure:

- Section 3: CI/CD Methodology
- Section 4: DevOps Process
- Section 5: Challenges & Solutions & Evaluation
- Sections 6-11: Deep dives on Secrets, Snyk+DAST+ASVS, Misconfigs, CD Security, IAM
- Section 12: Five Gap Analysis

- Section 13: Modern 2026 Blueprint

3. CI/CD Methodology: With Security vs. No Security

- With security (DevSecOps), security is a shared, automated responsibility woven into every pipeline stage with SAST at commit, SCA at build, DAST at test, IaC scan at deploy, and policy-as-code gates, whereas without security, security is ignored, manual, or bolted on at the very end with no scanners and no gates, and with security vulnerabilities are caught early, cheaply, and continuously while without security flaws ship straight to production where fixes cost 100x more (Singh, 2020), and the pipeline comparison shows that with security (DevSecOps) Commit leads to Build leads to Test with SAST, SCA, and DAST underneath feeding into Release (safe) and then Deploy plus Monitor leading to Trust and compliance, whereas with no security Commit leads to Build leads to Test with nothing, nothing, and nothing underneath leading to Release (vulnerable) and then Deploy leading to BREACH and then Fines, downtime, and lost trust, and the bottom line is that security in CI/CD is shift-left plus automated plus continuous while no security is shift-right or never plus manual plus reactive and that gap is the difference between resilience and disaster.

. DevOps Process Outlined: With vs. Without Security Challenges

- DevOps outlines a continuous infinite process with security embedded at every stage as DevSecOps. When security is missing, each phase introduces its own attack surface.

• **Phase**	• **WITH Security (DevSecOps)**	• **WITHOUT Security (Fragile DevOps)**
• **PLAN**	• Threat modeling (STRIDE), risk analysis	• No risk analysis, no threat model
• **CODE**	• Pre-commit hooks, Gitleaks, secrets scanning	• Hardcoded creds, .pem keys in repo

• Phase	• WITH Security (DevSecOps)	• WITHOUT Security (Fragile DevOps)
• **BUILD**	• SAST (Semgrep), SCA (Trivy/Snyk), signed images (Cosign)	• Untrusted, unsigned artifacts
• **TEST**	• DAST (ZAP), pen tests, fuzzing	• Functional tests only
• **RELEASE**	• Compliance gates (OPA), SBOM (Syft), audit trail	• No audit trail, no SBOM
• **DEPLOY**	• IaC scan (Checkov), Policy-as-Code (OPA/Sentinel)	• Misconfigured infra (public S3, open SG)
• **OPERATE**	• Runtime protection (Falco), WAF, least privilege IAM	• No runtime defense, admin keys
• **MONITOR**	• SIEM, CloudTrail, GuardDuty, alerts, threat hunt	• No visibility, no logging

- **Bottom Line:** A DevOps process without security isn't faster — it's just fragile; every phase silently accumulates risk until production becomes the breach point.

5. DevOps Adoption: Challenges, Solutions, and Evaluation

six core blockers to DevOps adoption are cultural resistance where Dev versus Ops versus Security silos, blame culture, and fear of automation exist, tooling sprawl where too many tools, no interoperability, and toolchain fragmentation occur, skill gaps where lack of IaC, container, and security expertise exists, security friction where security is seen as blocker not enabler causing bypass, legacy systems where monoliths, manual deployments, and incompatibility with pipelines exist, and scaling pains where what works for one team fails for 100 teams, and each challenge is human, technical, or organizational and none can be solved by tools alone, and unaddressed they stall

transformation, frustrate teams, and silently kill DevOps initiatives, while proven solutions battle-tested by elite performers per DORA that directly counter each challenge are blameless culture with psychological safety, post-mortems without blame, and shared ownership countering cultural resistance, platform engineering with Internal Developer Platform with Backstage, golden paths, and self-service countering tooling sprawl, upskilling with dojos, pair programming, and security champions program countering skill gaps, DevSecOps shift-left embedding security in IDE and pull request

not at end countering security friction, incremental modernization with Strangler Fig pattern not big bang rewrite countering legacy, and IaC plus automation with Terraform, Ansible, GitOps, and everything as code countering scaling pains, and applied together they convert blockers into enablers of speed, safety, and scale, and without measurement DevOps is hope-driven engineering while with measurement it becomes compounding advantage, where DORA metrics as the gold standard include deployment frequency measuring how often to production, lead time for changes measuring commit to production, mean time to recovery measuring how fast to recover from failure, and change failure rate measuring percentage of deployments causing failure, plus additional measures including SLOs and SLIs for Service Level Objectives and Indicators for reliability, feedback loops from monitoring to planning in less than one hour, and continuous improvement with Kaizen and weekly retrospectives with data, and they reveal whether speed, quality, and reliability are actually improving or just feeling better, while without DORA, SLOs, or feedback loops, deploy frequency, lead time, MTTR, and failure rate all remain unknown so teams cannot prove progress, spot regressions, or justify investment and every decision is guesswork resulting in blind, unverifiable DevOps that stalls, regresses, and eventually loses leadership trust, and with evaluation there is continuous improvement that is measured, data-driven, and compounding sprint after sprint while without evaluation it is stalled, regressed, and unverifiable with effort without evidence and evidence without effort, and the difference is not tools or talent but the discipline to measure what matters.

6. Secrets and Configuration Management

With Tool Integration:

Integrated secrets + config management centralizes, encrypts, rotates, and audits every credential and setting across environments. Tools: HashiCorp Vault, AWS Secrets Manager, SOPS, External Secrets Operator. Every secret is versioned, short-lived, and auditable. Configuration is templated via Helm/Kustomize, drift detected via ArgoCD.

Without Tool Integration:

No tool integration scatters secrets into code, chats, spreadsheets, and .env files — creating drift, leaks, and unmanageable deployments. Mangla (2023) found hardcoded john.pem, mary.pem, charles.pem in /.ssh/keys/ — a classic breach pattern. Difference is trust: one is secure, reproducible, and compliant; the other is a breach waiting to happen.

7. Layered Continuous Security: Snyk + DAST + ASVS

Snyk: Finds known flaws in code and dependencies before deployment via SCA. Cross-references NVD, provides fix PRs.

DAST: OWASP ZAP attacks the running app to expose runtime weaknesses that SAST cannot see: broken auth, misconfigurations, server errors.

ASVS (Application Security Verification Standard): OWASP framework providing requirements to verify coverage and rigor. It answers: "How much security is enough?" Levels 1-3.

Together: Snyk + DAST + ASVS deliver continuous, layered security: Snyk finds known flaws pre-deploy, DAST finds runtime flaws, ASVS provides the checklist to prove coverage.

Without Continuous Security: These tools are missing or disconnected — code ships with unpatched vulnerabilities, runtime flaws go undetected until exploited, and there is no standard to prove security is actually working. Difference: continuous security is automated, measurable, and shifts left; no continuous security is manual, unverifiable, and reactive — gap between resilience and breach.

8. Misconfiguration Detection: The Silent Killer

Most cloud breaches stem from settings not zerodays.

With Integrated Tooling:

Integrated detection catches misconfigs in IaC, containers, and cloud before deployment via Checkov, Trivy, Snyk IaC, and CSPM (Prowler, Security Hub).
Example: block_public_acls=true enforced via OPA before terraform apply.

Without Tool Integration:

Misconfigs slip through manual reviews, drift silently in production, and surface only during incidents or audits — when damage is already done. Mangla found 155 such cases: S3 public read, SG open all ports, IAM wildcard. Difference: integrated detection is proactive, automated, compliant; no integration is blind, reactive, breach-prone.

9. Continuous Deployment Security Concerns and Implementation

Core Concerns (Ullah et al., 2017; Mohan et al., 2018):

- Unsigned artifacts → supply chain risk
- Exposed secrets → credential theft
- Over-privileged credentials → privilege escalation
- Supply chain risk → poisoned dependencies
- Missing gates → no compliance check

With Secure CD Implementation:

- Signing: Cosign, Sigstore, SLSA attestation
- Secret Management: Vault, short-lived OIDC tokens, no static AWS keys
- Least Privilege: IAM roles per service, not admin
- Scanning: Trivy image scan, SBOM generation
- Policy Enforcement: OPA gates, mandatory approvals

With No Implementation:

None of these controls exist: pipelines auto-deploy unchecked code with admin keys and zero audit, making every release a direct path to compromise. Difference: secure CD is fast and safe; no implementation is fast and fragile — speed without guardrails is just accelerated risk.

10. IAM Best Practices and Future Work

IAM Best Practices (Battina & Sindhu, 2017; Mangla Table 2):

Narrow scope — least privilege at the most specific resource level, using just-in-time (JIT) elevation and service-specific roles instead of broad Owner/Contributor assignments.

Example: Instead of s3: on , allow s3:GetObject on arn:aws:s3:::prod-blog/* with condition aws:RequestedRegion=us-east-1.

No Future Work = Privilege Debt:

"No future work" means ignoring access lifecycle management: permissions granted for one task persist indefinitely, roles drift from actual job functions, dormant accounts become silent backdoors. Mangla's finding of IAM wildcard policy aws-goat-instance-boundary-policy is example.

Difference: narrow scope limits breach impact and stays auditable; no future work accumulates privilege debt until a single compromised credential unlocks everything.

11. Checklist-Based Security: CIS, STRIDE, and Continuous Monitoring

DevSecOps Avoids Misconfigurations by Embedding Checklists:

DevSecOps embeds known checklists (CIS Benchmarks, OWASP Top 10, ASVS) and automated scanners (IaC, container, CSPM) into the pipeline, so flaws are caught before deployment. Mangla used CIS + AWS best practices to detect 9 Critical, 11 High.

Without List or Evaluation:

No baseline of what to check, no automated detection, no scoring — misconfigurations remain invisible until attacker or auditor finds them.

12. Five Critical Research Gaps - Deep Dive

This section expands the core review's final page, which is the most valuable contribution.

Gap 1: CIS Baseline vs. No Detection Rules for Other Services

CIS Benchmarks: Provide hardened, industry-standard baselines for OS, cloud, containers, apps. Example: CIS AWS 1.4 has 50+ checks (MFA, password policy, CloudTrail enabled). Prowler and Security Hub implement them.

The Gap: CIS only covers what it explicitly defines. If no detection rules exist for custom apps, niche SaaS, internal tools, those services run unmonitored and unchecked. Example: CIS covers S3 but not a custom internal Flask app's misconfigured CORS.

Implication: CIS secures the known; everything else becomes silent blind spot. Solution: Extend CIS with custom OPA policies and Checkov custom checks for org-specific services.

Gap 2: STRIDE, Risk Analysis vs. No Tool-Based Detection

STRIDE: Spoofing, Tampering, Repudiation, Info Disclosure, DoS, Elevation structured threat modeling at design time. Tools: Microsoft Threat Modeling Tool, IriusRisk. Risk analysis prioritizes via DREAD.

The Gap: Without tool-based detection, threats stay theoretical no scanner, no runtime monitor, no alert validates whether risk is real. Example: STRIDE identifies Elevation of privilege via IAM wildcard" but without Semgrep/Checkov, no tool ever checks for in policy.

Implication: Analysis without detection is paperwork; detection without analysis is noise. Need both: STRIDE to identify, Semgrep/Checkov/Falco to detect.

Gap 3: Tool Integration in CI/CD vs. No Misconfig Detection

Integrated: Scanners (Checkov, Trivy, Snyk, CSPM) into CI/CD catch misconfigs automatically at every commit, build, deploy. Fail fast in PR.

No Detection: Pipelines push unchecked IaC, containers, cloud configs straight to production. Mangla's non-secure pipeline = 0 detections.

Gap: Integrated pipelines gate flaws pre-deploy; unintegrated pipelines gate nothing. This is the core RQ2 answer.

Gap 4: Secure vs. Non-Secure CD vs. Misconfigs Not Defined

Secure CD: Enforces signed artifacts, least-privilege deploys, policy gates, rollback all built on defined set of misconfigurations to block. Requires definition: Public S3 is misconfig.

Non-Secure CD: Auto deploys unchecked code, but deeper problem is misconfigs are not even defined so nothing can be gated. You cannot secure what you have not named; undefined misconfigs mean undefined security.

Solution: Define orgs misconfig dictionary

Gap 5: Security Monitoring Solution vs. Only Theoretical

Real Solution: SIEM (Splunk, Sentinel), CSPM (Prowler, Wiz), runtime defense (Falco, GuardDuty), alerting (PagerDuty) detects, correlates, responds continuously.

Only Theoretical: Monitoring exists on slides and diagrams no tools deployed, no alerts firing, no incident response. Many papers (Sojan et al., 2021) discuss monitoring theoretically but don't implement.

Gap: Theoretical monitoring protects nothing; deployed monitoring protects everything it sees. Mangla deployed CloudTrail; Singh recommends Falco/Sysdig.

13. Modern Blueprint 2026: Synthesis

Merging all gaps, we propose 2026 pipeline (detailed in separate blueprint file):

Stages:

1. Pre-commit: Gitleaks
2. CI: Semgrep (SAST), Trivy FS (SCA), Checkov + Trivy Config (IaC), OPA (PaC) - all parallel
3. Build: Trivy Image, Syft SBOM, Cosign sign, Kube-bench
4. Staging: ZAP + Nuclei DAST
5. Post-deploy: Prowler + Security Hub
6. Runtime: Falco + CloudTrail + GuardDuty
7. Reporting: Grafana DORA + Security metrics